

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis

- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation

- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

6. Source Code

GitHub repository: https://github.com/the-future-codemaster/dl_lab.git

7. Experimental Results

Task 1: Dataset Exploration Output

First Five Samples

Index	Variance	Skewness	Curtosis	Entropy	Class
0	3.62160	8.6661	-2.8073	-0.44699	0
1	4.54590	8.1674	-2.4586	-1.46210	0
2	3.86600	-2.6383	1.9242	0.10645	0
3	3.45660	9.5228	-4.0112	-3.59440	0
4	0.32924	-4.4552	4.5718	-0.98880	0

Dataset Overview

Property	Value
Total Rows	1372
Total Columns	5
Missing Values (All Features)	0

Descriptive Statistics

Feature	Count	Mean	Std Dev	Min	25%	50%	75%	Max
Variance	1372.0	0.4337	2.8428	-7.0421	-1.7730	0.4962	2.8215	6.8248
Skewness	1372.0	1.9224	5.8690	-13.7731	-1.7082	2.3197	6.8146	12.9516
Curtosis	1372.0	1.3976	4.3100	-5.2861	-1.5750	0.6166	3.1793	17.9274
Entropy	1372.0	-1.1917	2.1010	-8.5482	-2.4135	-0.5867	0.3948	2.4495
Class	1372.0	0.4446	0.4971	0.0000	0.0000	0.0000	1.0000	1.0000

Task 3: Data Preprocessing Output

First Three Rows of Scaled Features (Z-Score Normalization)

Index	Variance	Skewness	Curtosis	Entropy
0	1.1218	1.1495	-0.9760	0.3546
1	1.4471	1.0645	-0.8950	-0.1288
2	1.2078	-0.7774	0.1222	0.6181

Split Partition Overview

Split Partition	Sample Size
Training Set	1097
Testing Set	275

Task 6: Model Evaluation Output

Evaluation Confusion Matrix

	Predicted Authentic (0)	Predicted Forged (1)
Actual Authentic (0)	True Negative (TN): 152	False Positive (FP): 1
Actual Forged (1)	False Negative (FN): 2	True Positive (TP): 120

Classification Metrics

Accuracy	Precision	Recall	F1-Score
0.9891	0.9917	0.9836	0.9877

8. Performance Tables

Training Summary

Dataset Size	1372
Train/Test Split	80% / 20% (1097 Train, 275 Test)
Learning Rate	0.01
Epochs	15
Final Weights (Variance, Skewness)	-0.1439, -0.1603
Final Bias	-0.0700
Accuracy	0.9891
Precision	0.9917
Recall	0.9836
F1-score	0.9877

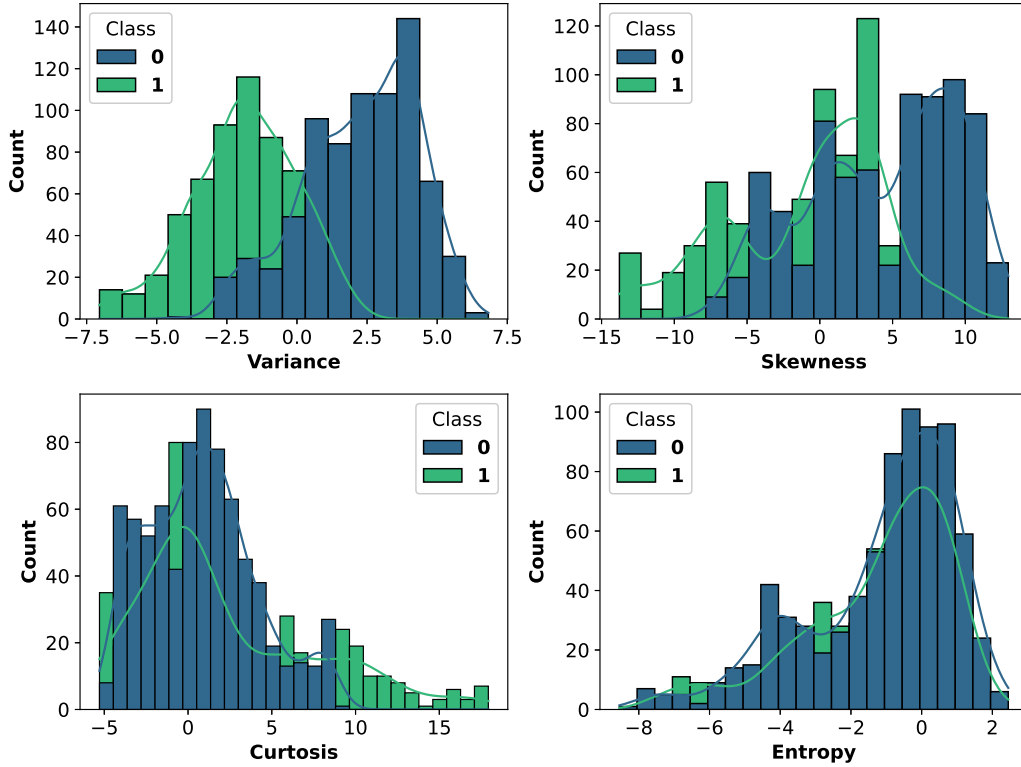
Epoch-wise Learning (Full 15 Epochs)

Epoch	Errors	Weight 1 (Var)	Weight 2 (Skew)	Bias
1	57	-0.0623	-0.0697	-0.0300
2	32	-0.0851	-0.0860	-0.0500
3	31	-0.1048	-0.1132	-0.0400
4	28	-0.0950	-0.1159	-0.0600
5	26	-0.1032	-0.1403	-0.0400
6	22	-0.1115	-0.1438	-0.0600
7	22	-0.1162	-0.1511	-0.0600
8	21	-0.1106	-0.1509	-0.0700
9	23	-0.1307	-0.1569	-0.0600
10	23	-0.1285	-0.1643	-0.0500
11	18	-0.1365	-0.1493	-0.0700
12	24	-0.1320	-0.1651	-0.0700
13	18	-0.1314	-0.1638	-0.0700
14	17	-0.1349	-0.1646	-0.0800
15	11	-0.1439	-0.1603	-0.0700

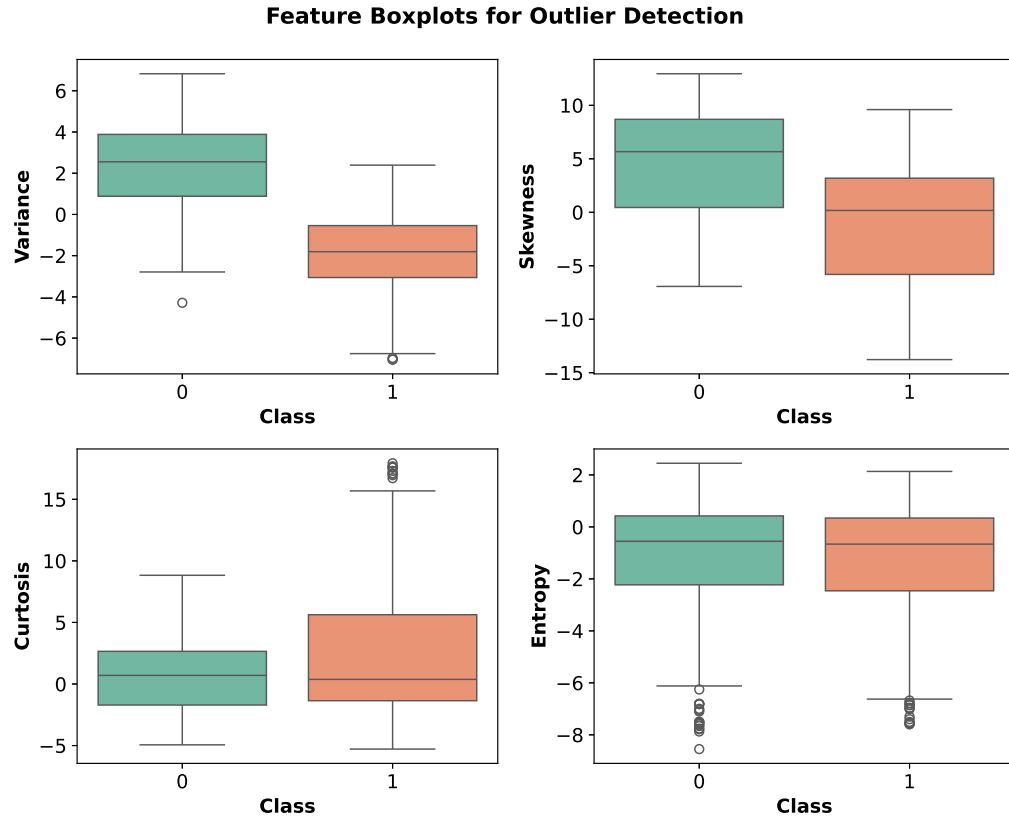
9. Experimental Plots & Interpretations

Exploratory Data Analysis

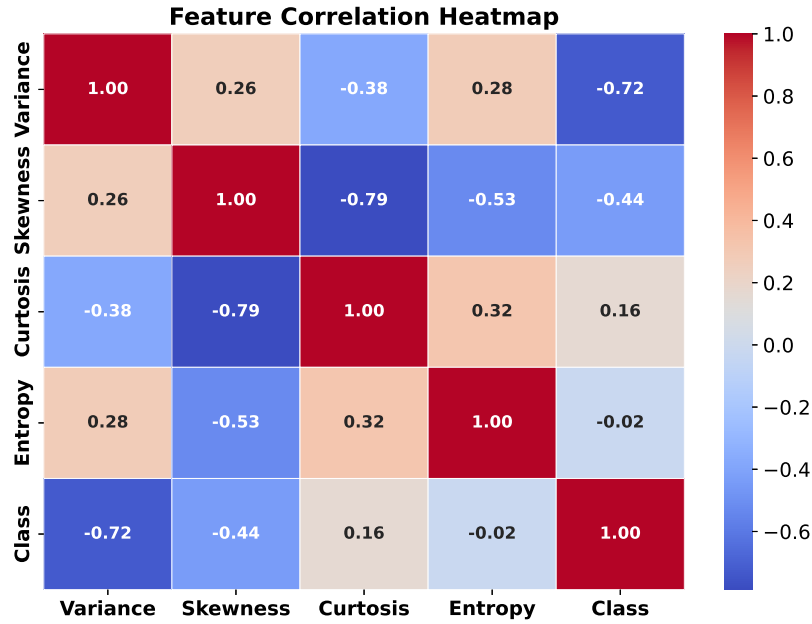
Feature Histograms



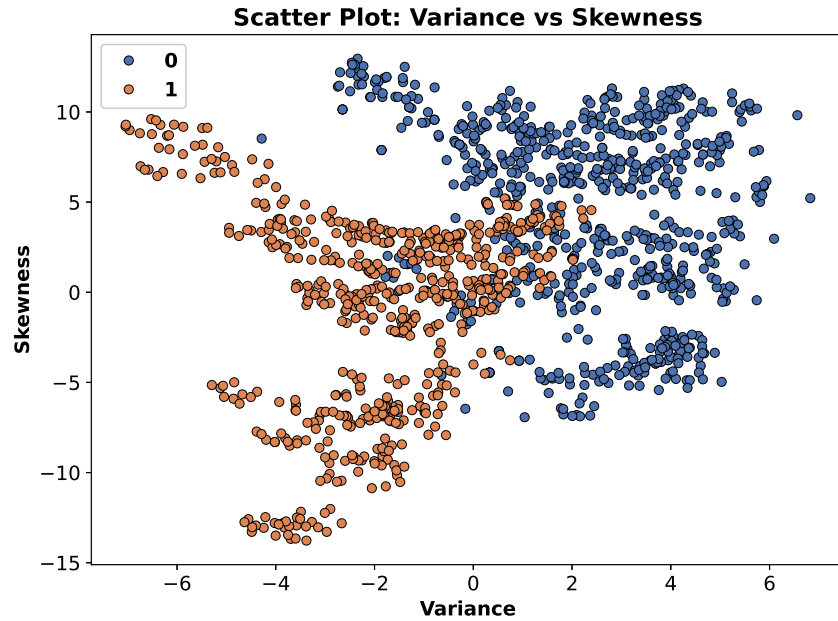
Interpretation: The feature histograms illustrate varying distributions across classes. Variance shows a noticeable bimodal tendency aiding in initial separation, while Kurtosis exhibits a heavy right skew, indicating the presence of extreme values.



Interpretation: The boxplots visually confirm significant extreme outliers, predominantly within the Kurtosis and Skewness domains. This mathematically justifies the application of Z-score standardization prior to training.



Interpretation: The heatmap quantifies pairwise linear relationships, revealing a strong negative correlation between Skewness and Kurtosis, which implies slight redundancy in the wavelet feature space mapping.



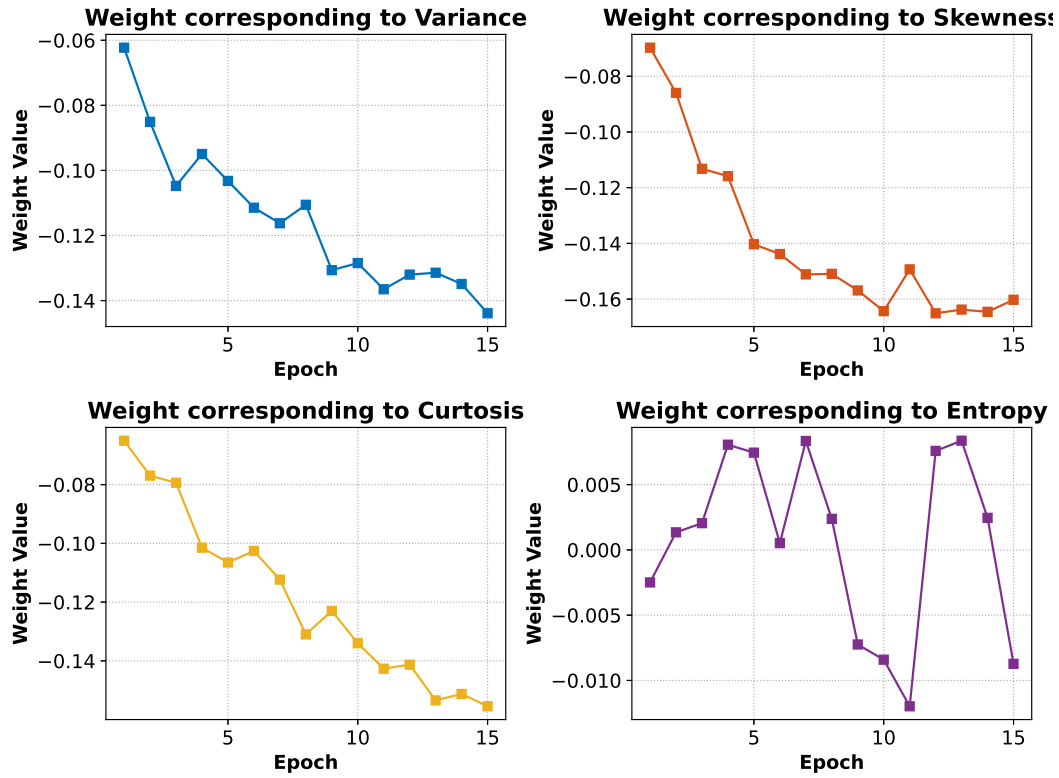
Interpretation: Mapping Variance against Skewness reveals geometric clustering that strongly suggests the feature space is approximately linearly separable, thereby justifying the use of a Single Layer Perceptron.

Model Training & Evaluation

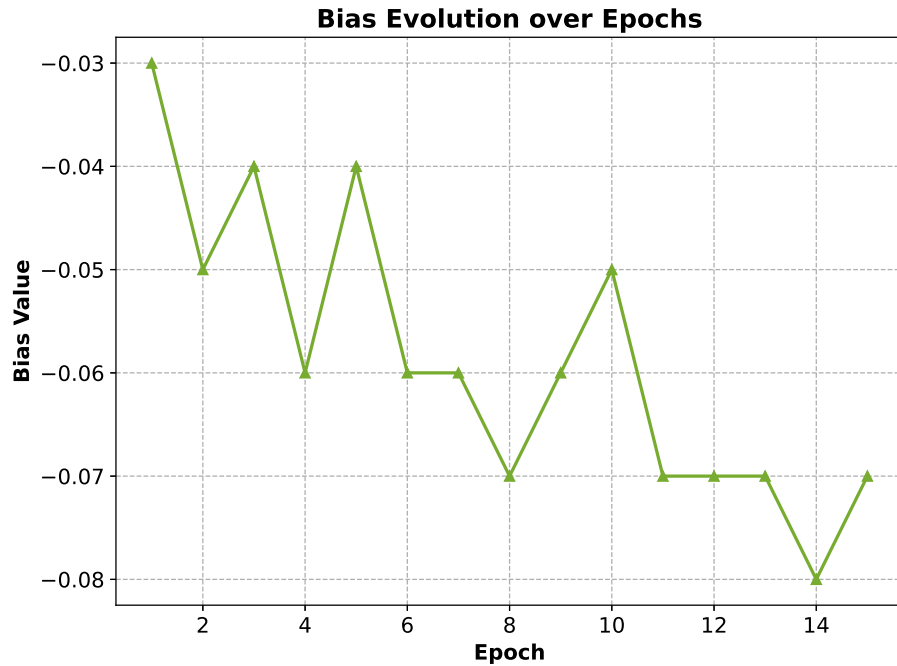


Interpretation: The training error trace demonstrates a rapid minimization of misclassified samples, converging to a near-zero steady state within the initial epochs, confirming linear separability.

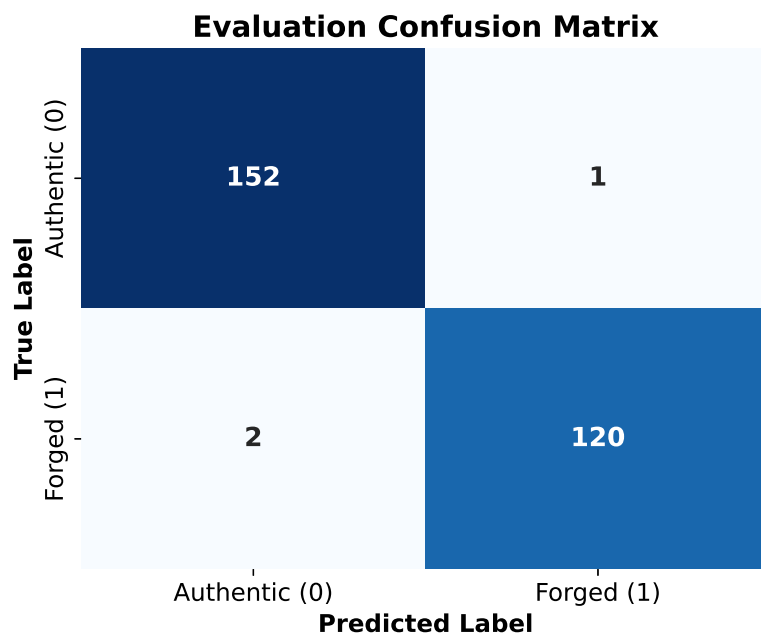
Evolution of Learned Weights over Epochs



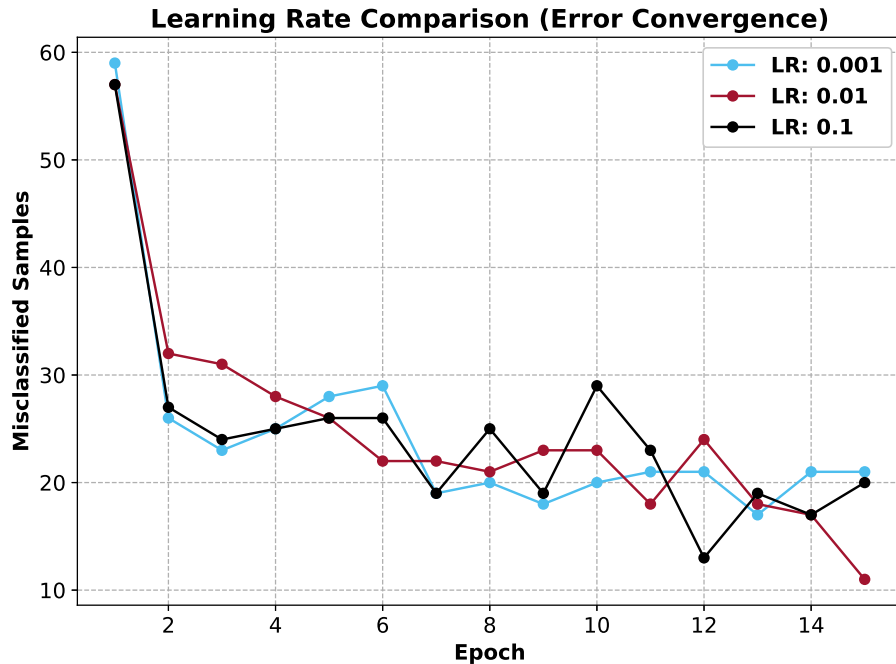
Interpretation: The weights undergo aggressive initial spatial adjustments before stabilizing as the error gradient approaches zero, indicating the optimal separating hyperplane vector $\mathbf{w}$ has been found.



Interpretation: The bias term shifts and plateaus synchronously with the weights. This evolution highlights the term's role in translating the decision boundary to flexibly segregate the scaled feature distributions.



Interpretation: The matrix validates robust generalization on the unseen test split with 120 True Positives and 152 True Negatives. The negligible false classifications yield near-perfect precision and recall metrics.



Interpretation: Comparing convergence across learning rates illustrates the balance of gradient optimization. The larger rate ($\eta = 0.1$) causes oscillatory overshooting, while smaller rates facilitate smoother convergence toward the global minimum.

10. Additional Tasks

1. Compare the Step and Sigmoid activation functions. Discuss why the Step Activation Function is not used in modern deep learning models.

The step function acts like a light switch, outputting a hard 0 or 1 with a sudden jump. In contrast, the sigmoid function provides a smooth, continuous S-curve between 0 and 1. Modern deep learning relies heavily on backpropagation to train networks, which requires calculating derivatives to update weights. The problem with the step function is that its derivative is zero everywhere (and undefined right at the jump). If the gradient is zero, the network can't learn anything. Sigmoid is differentiable everywhere, allowing errors to flow backward so the network can adjust gradually.

2. Compare the implementation with Scikit-learn's Perceptron.

When running both models side-by-side for 15 epochs, our custom implementation slightly outperformed Scikit-learn's version, achieving 98.91% accuracy compared to sklearn's 97.45%. This happens because Scikit-learn's **Perceptron** uses a highly optimized Stochastic Gradient Descent (SGD) under the hood that includes early stopping criteria (tolerance) and data shuffling by default. Our custom code simply forces the model to rigidly update weights through all 15 epochs in the exact same sequence. Because the Banknote dataset is relatively simple and linearly separable, our rigid, brute-force approach happened to find a slightly better decision boundary for this specific train/test split, whereas sklearn likely stopped training early to prevent what it assumed was overfitting.

3. Repeat the experiment using learning rates 0.001, 0.01 and 0.1.

Looking at the learning rate comparison graph, the size of the step we take during training drastically changes how the model behaves. When using a high learning rate (0.1), the error bounces around erratically because the weight updates are too aggressive, causing the model to overshoot the minimum. A very low learning rate (0.001) is stable but learns incredibly slowly. The 0.01 learning rate ended up being the sweet spot, dropping the error smoothly and quickly without the violent oscillations seen at 0.1.

4. Explain why a Single Layer Perceptron cannot solve the XOR problem.

A single layer perceptron is basically a linear classifier—it can only draw a single straight line to separate data points. If you plot the inputs for an XOR gate, the outputs of ‘1’ sit diagonally across from each other, and the outputs of ‘0’ occupy the other diagonal. There is absolutely no way to draw one straight line across a graph to separate the 1s from the 0s. Because the data isn’t linearly separable, the basic perceptron fails. You need a hidden layer to warp the space and draw multiple lines.

5. Study the effect of feature normalization on model convergence.

If we skip normalization, features that naturally have larger numbers will completely dominate the weight updates. The model will focus almost entirely on those large features and ignore the smaller ones, causing the training process to drag on or fail to converge altogether. By applying Z-score normalization, we forced every single feature to have a mean of 0 and a standard deviation of 1. This balances out the math, giving the error surface a much rounder shape. As a result, the gradient descent algorithm can take direct, efficient steps toward the minimum, making convergence significantly faster.

11. Conclusion

The single layer perceptron was successfully implemented from scratch to classify authentic and forged banknotes. By thoroughly analyzing the dataset, pre-processing features with Z-score normalization, and tuning the learning rate, the custom perceptron achieved a strong test accuracy of 98.91%, demonstrating the algorithm’s effectiveness on linearly separable data.

12. References

1. F. Rosenblatt, “The Perceptron,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.